

Searching Binary Embeddings

Calculations and Experiments

12 September 2026

Abstract

This report compares clustered scanning, Bitplanes and Backward Walk on one collection of binary documents. The calculations explain how cluster size, bit splits and key lookups affect work and memory. Experiments check those estimates and measure when each method is faster under the same recall and RAM requirement.

1 Introduction

1.1 Exa method and end to end pipeline

Exa describes a search pipeline that shortens Matryoshka embeddings, stores document values as binary signs, selects nearby clusters, and scores their documents with the floating-point query [1]. The results can then be reranked with uncompressed vectors. We refer to that approach as the original method, or A. The experiments use our implementation of this search stage; they do not measure Exa’s production system.

Documents are indexed before queries arrive. At query time, A opens some clusters and scores their documents. The blue box is the step that B and C change.

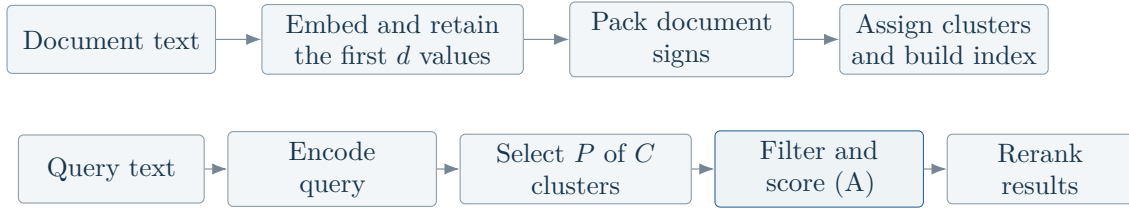


Figure 1: Exa’s described end-to-end flow. B and C replace the blue search step. The benchmark starts from cached embeddings and times cluster selection and search.

1.2 Idea background

The query itself tells us the best possible combination of document signs. To follow the arithmetic, consider a five-value example:

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

A document bit of 1 represents +1; a bit of 0 represents −1. At the first position, $0.7 \times (+1) = 0.7$ is better than $0.7 \times (-1) = -0.7$. At the third position, $-0.4 \times (-1) = 0.4$ is better than $-0.4 \times (+1) = -0.4$. Following this rule at every position gives 11001, with score $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$.

A mismatch reduces the score by twice the size of that query value. Mismatching the first position loses 1.4; mismatching the last loses 0.6. In general,

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

Here p adds the sizes of the query values at mismatched positions. For example, 11000 differs only at the last bit, giving score $2.5 - 2(0.3) = 1.9$.

The question is whether an index can find documents near the preferred pattern without scoring every document. Bitplanes use the query values to choose which bits to split on. Backward walk starts with a short preferred pattern and tries other patterns when needed. The preferred pattern may not exist in the collection, so both methods must retain alternatives.

1.3 Exploring two new methods: “Bitplanes” and “Backward Walk”

1.3.1 Bitplanes

Method B keeps the packed rows and adds a second arrangement of their signs. Each *bitplane* stores one bit position across all documents in a cluster. The figure shows three documents with the query from Section 1.2. A candidate mask records which local rows remain.

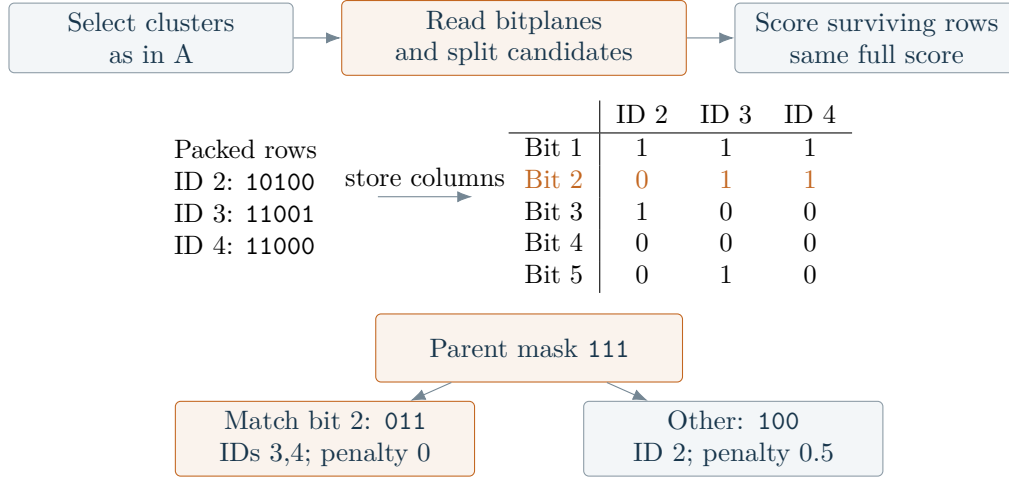


Figure 2: Orange marks the changed stage and the selected plane. Both child masks retain all three positions. On a larger cluster, this physical width is a major cost.

The absolute sizes of the query values give this position order: 1, 4, 2, 3, 5. For a positive query value, a parent mask A is split into $A \& B_i$ and $A \& \neg B_i$. The first child matches; the other adds $|q_i|$ to its penalty. Negative query values reverse the roles. Alternatives remain in a priority queue, ordered by known penalty, because a preferred early bit does not determine the final ranking.

Equation 1 gives a safe upper bound: a branch with known penalty p can score at most $Q - 2p$. If IDs 3 and 4 score 2.5 and 1.9, the alternative with penalty 0.5 has bound 1.5 and cannot improve the top two. Equal-score cases retain the common document-ID tie rule.

```
bitplanes(query, clusters, K, leaf_size, node_budget):
    table, best = prepare_score_table(query), empty_top_k(K)
    order = positions_by_decreasing_abs_value(query)
    queue = full candidate mask for each selected cluster
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): break
        if node.count <= leaf_size or no_bits_remain(node):
            score_set_documents(table, node.mask, best)
        else:
            queue_nonempty_children(split(node, order))
    return best.sorted_by_score_then_id()
```

The leaf size decides when to stop splitting and score the remaining rows. Unlimited search with safe bounds is exact within opened clusters; a finite work limit can lose results.

1.3.2 Backward Walk

Method C forms a short key from a chosen set of h bit positions. In each cluster, documents with the same key are stored consecutively. A directory maps an occupied key to the start and length of that row range. Search begins at the query's preferred key and enumerates alternatives in increasing weighted mismatch penalty.

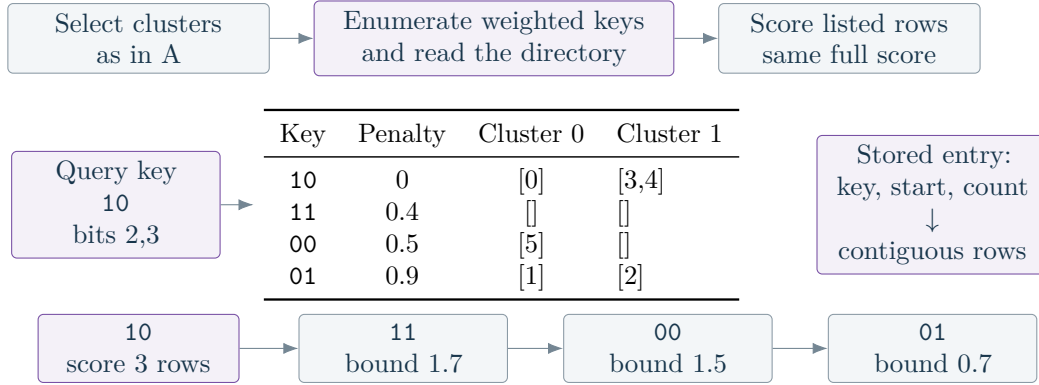


Figure 3: Purple marks the changed stage. Here the first key returns IDs 0,3,4. Their top two scores are 2.5 and 1.9, so the next key's bound 1.7 already permits stopping. The remaining entries show the order that a query requiring more work would follow.

The ordering uses the sum of absolute query values at the flipped positions. Counting flipped bits alone would give the wrong order when query weights differ. There is one shared enumeration queue across opened clusters, followed by a lookup in each cluster's directory. Empty lookups also take time. Removing a key constraint is not a single ordinary hash lookup: the newly admitted patterns must be enumerated or supported by a different data structure.

```
backward_walk(query, clusters, K, key_positions, limits):
    table, best = prepare_score_table(query), empty_top_k(K)
    keys = unique_keys_by_increasing_penalty(query, key_positions)
    for key, penalty in keys:
        if cannot_improve(penalty, query, best): break
        for cluster in clusters:
            if work_limit_reached(limits):
                return best.sorted_by_score_then_id()
            rows = cluster.directory.lookup(key)
            score_each_row(table, rows, best)
    return best.sorted_by_score_then_id()
```

A short key omits some bit positions, so every retrieved row still receives the complete score. Finding enough candidates does not by itself prove sufficient recall. Exhausting the keys or ruling out every remaining bound is exact within opened clusters; stopping at a candidate or lookup budget is approximate. Neither B nor C can recover a document excluded by the initial routing.

2 Preliminary calculations

The collection contains $N = 1,000,000$ documents. The same 200 query vectors, 256-bit codes and reference answers are used throughout. Each method may choose its own cluster count C , probe count P and local settings. A probe opens one cluster. The result count is $K = 100$; d denotes the number of document bits.

2.1 Time complexity

The formulas count the work for one query. Converting it to milliseconds requires measured operation costs or direct stage timings. The final comparison uses measured query time, including cluster selection and local search.

2.1.1 Original method

If the opened clusters contain 200, 350 and 150 documents, A computes 700 complete scores. For cluster sizes n_c , the count is

$$F_A = \sum_{c \text{ opened}} n_c.$$

The approximation NP/C is useful only when the opened clusters are close to equal in size. The experiment counts their actual documents.

The scorer follows Exa’s four-coordinate lookup-table approach [1]. A 256-bit code requires 64 table terms. Each term extracts four signs, reads an entry and adds it to the score. Storing a document in four machine words does not reduce scoring to four operations. For group width g , let $G = \lceil d/g \rceil$. Preparing the query table creates $G2^g$ entries; scoring F documents evaluates FG terms.

Define

$$T_0 = T_{\text{route}} + T_{\text{table}}, \quad T_{\text{score}}(F) = Fc_s + T_{\text{top}}(F, K).$$

Here c_s prices the work within a complete score; row-access costs can differ between methods. T_{top} covers retaining the best K results and ordering the output. Its worst-case work is $O(F \log K)$; not every score replaces a retained result. A’s total is $T_A = T_0 + T_{\text{score}}(F_A)$.

2.1.2 Bitplanes

A cluster of 6400 documents needs 100 words in each mask when a word holds 64 bits. If a split leaves only 80 candidates, its mask still has 100 words. The next split reads all 100 positions again.

Let $W_c = \lceil n_c/w \rceil$, with word width $w = 64$ here. If a query performs J_c splits and scores E_c final masks in cluster c , the word visits are

$$V_s = \sum_c J_c W_c, \quad V_l = \sum_c E_c W_c.$$

A split reads a parent and a bitplane and writes children. One word visit therefore includes several bit operations and memory accesses. Ordering query coordinates costs $O(d \log d)$; the branch queue adds work when candidate sets are inserted or removed.

A zero word in a final mask skips scoring. A nonzero word starts a sequence of row reads; let V_+ count these words. Reading 100 mostly empty words and reading 100 words containing many candidates have different costs. With F_B scored documents,

$$T_B = T_0 + T_{\text{order}} + c_b V_s + c_l V_l + c_r V_+ + T_{\text{score}}(F_B) + T_{\text{branches}}.$$

The coefficients price split-word work, final-mask checks and the additional cost of starting row reads. Actual splits and survivors depend on the query and stored signs. They cannot be obtained from N and RAM alone.

2.1.3 Backward Walk

Trying three keys in two clusters requires six lookups, including empty results. If four lookups find document ranges, there are four range starts. All rows in those ranges still receive complete scores.

For E fully processed keys, $H = PE$. Let H_+ count successful lookups and F_C count scored rows. Then

$$T_C = T_0 + T_{\text{keys}} + c_h H + c_p H_+ + T_{\text{score}}(F_C).$$

There is one shared key queue. T_{keys} includes ordering the h key coordinates and generating alternatives. Priority-queue enumeration has an $O(E \log(E + 1))$ upper bound. An empty lookup pays the lookup cost; a successful one also starts a document range. The experiment records lookup and score counts rather than assuming equal key occupancy.

2.2 Memory complexity

Stored index data and temporary query memory are separate. The running program, unused array capacity and allocation overhead also consume RAM. Construction may have a higher peak because input arrays and the new index coexist.

2.2.1 Original method

A 256-bit code occupies 32 bytes. An eight-byte ID brings the total to 40 bytes per row, or 40 MB for one million rows. With $u = w/8$ bytes per packed word,

$$M_{\text{rows}} = N(u \lceil d/w \rceil + b_{\text{id}}).$$

Cluster storage is additional. For example, C floating-point centroids of width d_R require $4Cd_R$ bytes for coordinates, plus labels and row ranges.

A query also needs its vector, score table, selected cluster IDs and top- K results. The current query vector occupies 1024 bytes; its 64 groups of 16 double-precision table entries occupy 8192 bytes. These are data sizes, not the complete process footprint.

2.2.2 Bitplanes

B retains the rows and adds one bitplane per coordinate:

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil.$$

One global cluster adds 32 MB of planes to the 40 MB of rows and IDs. Separate clusters introduce padding because each plane ends on a whole-word boundary.

Waiting branches retain masks. If $A_c(t)$ masks for cluster c exist at time t , their peak data size is

$$M_{\text{masks}} = u \max_t \sum_c A_c(t) W_c.$$

This includes a parent while both children are created. Queue entries and unused capacity are additional. Counting only the mask currently being processed is insufficient.

2.2.3 Backward Walk

C stores rows consecutively within each key range. Their existing IDs serve as the document lists, so a second N -element ID array is unnecessary. Each occupied entry stores a key, a start position and a row count. For U entries,

$$M_{\text{entries}} = U(b_{\text{key}} + 2b_{\text{offset}}).$$

A four-byte key and two eight-byte positions require 20 bytes of field data per entry. Hash-table pointers and allocation overhead are additional. Query memory includes the key queue. More key bits can increase directory size without reducing the final scoring work.

2.3 Recall estimation

Returning 99 of the reference top 100 IDs gives 99% recall. The reference is the exact ranking under the unchanged full binary-vector score, with the same ID rule for ties. This measures agreement with that ranking, not whether the text answers a person's question.

Let G_q be the reference IDs and $O_{a,q}$ the IDs returned by method a . Batch recall is

$$\bar{R}_a = \frac{1}{n_q K} \sum_{q=1}^{n_q} |G_q \cap O_{a,q}|, \quad n_q = 200.$$

This uses the saved IDs directly. It requires no assumption that document signs or embedding coordinates are independent.

2.3.1 Original method

Let D_q contain the documents in the opened clusters and $g_q = |G_q \cap D_q|$. A scores all of D_q , so its per-query recall is exactly $R_{A,q} = g_q/K$. Opening clusters containing 95 reference IDs gives 95% recall; scoring their other documents cannot recover the five reference IDs in unopened clusters.

2.3.2 Bitplanes

Among the g_q reference IDs that reached the clusters, let $r_{B,q}$ be the fraction returned. Then $R_{B,q} = (g_q/K)r_{B,q}$. For example, if eight of ten reference documents reach the clusters and six of those eight are returned, recall is $(8/10)(6/8) = 60\%$. Unlimited search with safe bounds makes $r_{B,q} = 1$. A finite node budget must be checked against the returned IDs. If $g_q = 0$, recall is zero.

2.3.3 Backward Walk

Similarly, $R_{C,q} = (g_q/K)r_{C,q}$. Visiting all necessary keys, or safely ruling out the remaining keys, makes the local fraction one. Candidate or lookup limits can reduce it. The batch value averages these per-query products; multiplying separately averaged fractions is generally incorrect. A key returning many rows establishes no recall level unless the required reference IDs are among them.

2.4 Summary

Each formula uses its method’s own opened clusters and local work. Let M_0 be allocated row and router storage, Q_0 common query memory, D_C allocated directory memory, and Q_B, Q_C extra masks and queues. Allocation overhead is included in these memory terms.

Time complexity

Method	Query-time formula
Original method	$T_0 + T_{\text{score}}(F_A)$
Bitplanes	$T_0 + T_{\text{order}} + c_b V_s + c_l V_l + c_r V_+ + T_{\text{score}}(F_B) + T_{\text{branches}}$
Backward Walk	$T_0 + T_{\text{keys}} + c_h H + c_p H_+ + T_{\text{score}}(F_C)$

T_0 includes routing and table preparation. Costs already included in a measured stage must not be added a second time. Query encoding and later reranking are outside this timer.

Memory complexity

Method	Stored index	Temporary query memory
Original method	M_0	Q_0
Bitplanes	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
Backward Walk	$M_0 + D_C$	$Q_0 + Q_C$

The process, including these arrays and the running program, must fit 32 GB. A peak that includes construction is reported as such, not as index size.

Recall estimation

Method	Per-query recall	Local exactness condition
Original method	g_q/K	Score all opened documents
Bitplanes	$(g_q/K)r_{B,q}$	Complete search with safe bounds
Backward Walk	$(g_q/K)r_{C,q}$	Visit or safely rule out remaining keys

The reported batch value averages the same 200 reference answers.

3 Deriving Optima

The objective is minimum measured retrieval time at a required batch recall and a 32 GB RAM limit. Only index and search parameters change. Document codes, query vectors and reference answers remain identical. Each method chooses its own cluster count and local settings.

3.1 Original method

For a given layout, the reference answers show exactly how many clusters A must open. Suppose a reference document belongs to the fifth cluster in the query’s routing order. A includes it when $P \geq 5$. Record this rank for every reference document and query.

Let $s_{qi}^{(C)}$ be the cluster rank of reference result i for query q . At recall requirement ρ , the smallest qualifying probe count is

$$P_A(C, \rho) = \min \left\{ P : \frac{1}{n_q K} \sum_{q=1}^{n_q} \sum_{i=1}^K \mathbf{1}[s_{qi}^{(C)} \leq P] \geq \rho \right\}.$$

The indicator contributes one when the document’s cluster is opened. With 200 queries and 100 reference IDs each, 99% recall requires at least 19,800 recovered IDs across the batch. These ranks are calculated before timing the searches.

The experiment uses one global cluster and learned layouts with 256, 4096 and 8192 clusters. For each layout it computes probe cutoffs for 80%, 90%, 95% and 99% recall, and also opens all clusters. Query time and selected-row counts are then measured. These cutoffs describe this query batch; they do not guarantee the same recall on future queries.

A derivative explains the competing costs. Suppose clusters are balanced, routing costs αC , each row costs approximately β , and P stays constant. Then

$$T_A(C) \approx \alpha C + \beta NP/C, \quad \frac{dT_A}{dC} = \alpha - \frac{\beta NP}{C^2}, \quad C^* = \sqrt{\frac{\beta NP}{\alpha}}.$$

This balances routing more clusters against scanning fewer rows. Under a recall requirement, $P_A(C, \rho)$ can change with C . The measured comparison uses that probe count and actual cluster sizes; the fixed- P derivative is not a complete solution.

3.2 Bitplanes

Consider a mask covering 6400 documents, or 100 words. If 200 candidates remain and a split removes 100, it saves 100 complete scores. It also adds 100 split-word visits and branch work. For ΔF avoided scores, the local comparison is

$$\Delta F c_s > W c_b + c_l \Delta V_l + \Delta T_{\text{branches}} + \Delta T_{\text{row starts}} + \Delta T_{\text{top}}.$$

The right side includes the changes caused by the extra split. Those changes need not all be positive. The complete search must also retain the required recall; scoring fewer rows by ending too early is insufficient.

If each split halves one candidate set, after j cuts it contains approximately $n2^{-j}$ rows. Ignoring changes in queue and row-start costs gives the guide

$$T_B(j) \approx T_0 + jW c_b + W c_l + n2^{-j} c_s, \quad j^* = \log_2 \left(\frac{n \ln(2) c_s}{W c_b} \right).$$

This is an illustrative cost balance, not a recall estimate for these embeddings. If two bitplanes are identical, splitting on the second after the first removes no additional rows. Alternative branches also add paths. The experiment measures these effects rather than assuming repeated halving.

B uses every cluster and probe option above, then varies local search: leaf size 128 with node budgets 256 and 1024; leaf size 512 with unlimited work; and leaf size N , which permits scoring an opened cluster without splitting. A node budget counts visited candidate sets, including leaves. Leaf sizes 32 and 128 are also compared at budget 4096. Actual splits, final-mask reads and scored rows explain the resulting time.

The probe cutoff computed for A is a necessary routing condition for B using the same layout. It may not suffice when local limits lose more results. Returned IDs determine whether each complete setting qualifies.

3.3 Backward Walk

The key must be wide enough to rule out some patterns. For the first saved query, the ideal full score is 12.361. Even if all eight key bits mismatch, the remaining full-score upper bound is 11.373. The 100th reference score is only 9.994. Thus even the least promising eight-bit key cannot be discarded using this bound. All 200 queries have this problem at key widths 1, 4, and 8.

At the same routing settings, changing key width from h to h' helps when

$$T_{\text{score}}(F_h) - T_{\text{score}}(F_{h'}) > c_h(H_{h'} - H_h) + c_p(H_{+,h'} - H_{+,h}) + T_{\text{keys},h'} - T_{\text{keys},h}.$$

A smaller first range does not determine the answer. Search may need many additional keys, including empty ones, before finding enough high-scoring documents or proving that remaining keys cannot improve the result.

The experiment tries keys of 1, 4, 8, 16, 20, and 24 bits. For all 200 queries, the short 1-, 4-, and 8-bit keys cannot be discarded using the score bound, so their complete key spaces are searched. Longer keys are compared with lookup limits of 4096 and 65,536. The full 256-bit score remains unchanged. Directory memory is measured for each index, and returned IDs establish recall. No $N/2^h$ occupancy assumption is used to assign recall or declare a winning key width.

3.4 Comparison

Let Θ_a contain method a 's tested settings. Its final choice is

$$\theta_a^* = \underset{\theta \in \Theta_a}{\operatorname{argmin}} T_{a,\text{measured}}(\theta) \quad \text{subject to} \quad \bar{R}_a(\theta) \geq \rho, \quad M_{a,\text{process}}(\theta) \leq 32 \times 10^9 \text{ bytes}.$$

Method	Adjustable settings	Work that may be reduced
Original method	Cluster count C and probes P	Documents reaching the scorer
Bitplanes	C , P , leaf size and node budget	Scores, at the cost of mask and branch work
Backward Walk	C , P , key width h and lookup limit	Scores, at the cost of lookups and range starts

The methods are compared after each has selected its own qualifying settings. Results below the recall target remain visible, but cannot establish a speedup at that target. This finds the best measured choices within the tested ranges on the unchanged query batch. It does not prove that every possible index configuration has been exhausted.

4 Experiments

4.1 Setup

Every run uses the same one million MS MARCO document embeddings [3], the same 200 query embeddings, and the same reference answers. The embeddings were already produced with Nomic [4]. We do not regenerate them, change their values, shorten them, or select different queries for different methods.

The documents contain 256 sign bits; queries retain 256 floating-point values. The reference answer is the best 100 document IDs from a full scoring calculation. A recall of 99% means returning 99 of those same 100 IDs on average. All methods use that identical reference.

Only the index parameters listed in Section 3 change: clusters, probes, B’s leaf size and branch budget, and C’s key width and lookup limit. The complete score still uses all 256 bits.

For each cluster count, the code finds probe counts reaching 80%, 90%, 95%, and 99% recall with the original scan. It also includes opening every cluster. These probe choices are available to A, B and C. This avoids giving A an unnecessarily expensive probe count when comparing it with a more selective method.

We measured 256 parameter settings in total.

The machine is an Apple M5 Max with 128 GiB of physical memory. Each process has a 32 GB memory budget and uses one CPU thread. Every setting searches all 200 queries twice. The fastest successful settings are then repeated in three processes, still using those same queries. Reported time is the middle measurement, or median.

Retrieval time includes choosing clusters and searching their documents. It excludes building the index and generating embeddings. File hashes for the document, query and reference arrays are checked before and after the measurements.

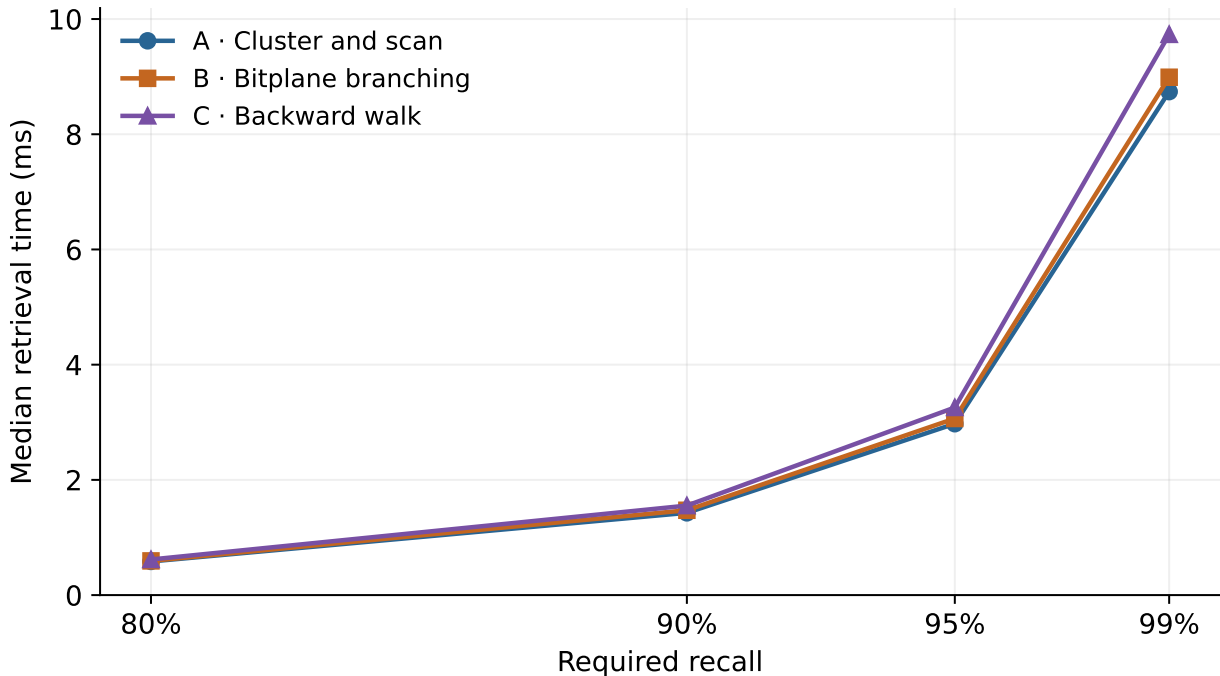


Figure 4: Time required at each recall target. All points use the same document and query arrays. Lower time is better; every plotted setting meets the recall requirement.

4.2 Results and calculation

Table 1: Repeated measurements at the same recall requirements.

Required recall	Method	Actual recall (%)	Median (ms)
80%	A	80.01	0.5842
80%	B	80.01	0.5936
80%	C	80.01	0.6181
90%	A	90.02	1.43
90%	B	90.02	1.47
90%	C	90.02	1.55
95%	A	95.00	2.97
95%	B	95.00	3.07
95%	C	95.00	3.26
99%	A	99.00	8.74
99%	B	99.00	8.99
99%	C	99.00	9.73

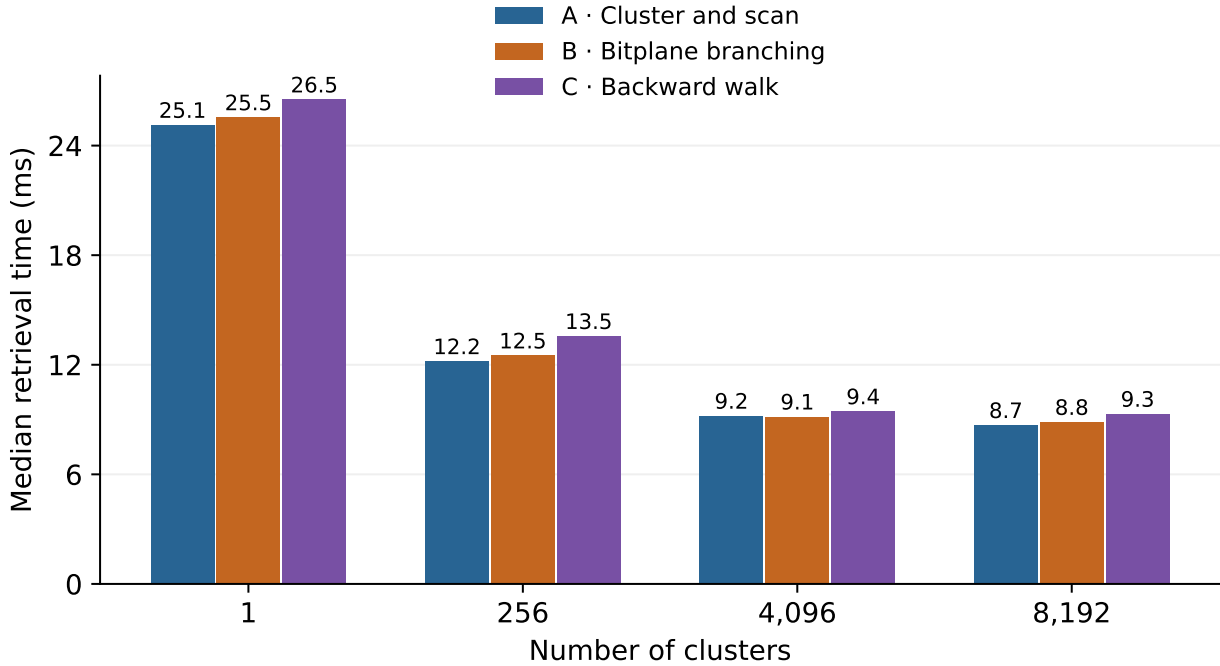


Figure 5: Initial measurements at each cluster count for 99% recall. Table 1 uses repeated runs of the selected settings, so small timing differences are expected. All inputs remain unchanged.

Table 2: Settings for 99% recall. L is leaf size and h is key width. Budget counts visited candidate groups for B and key lookups for C; zero means unlimited.

Method	Route	C	P	L	Budget	h	RAM (MB)
A	IVF	8,192	2,440	—	—	—	190.6
B	IVF	8,192	2,440	512	0	—	231.6
C	IVF	8,192	2,440	—	0	1	161.7

The following page connects those settings to their actual work. A name such as “Bitplanes” does not tell us whether the chosen setting split any candidates; a large leaf can make it score an opened cluster immediately.

4.3 What the numbers mean

Table 3: Average work per query for the selected 99% settings.

Method	Rows scored	Split words	Leaf words	Key lookups
A	295,858	0	0	0
B	295,858	0	5,820	0
C	295,858	0	0	4,880

At 99% recall, all three selected settings open 2440 of 8192 clusters and score the same 295,858 documents per query on average. B performs no splits and checks 5820 mask words. C uses a one-bit key and makes $2440 \times 2 = 4880$ lookups. Their median times are 8.74, 8.99 and 9.73 ms for A, B and C respectively. The extra index work avoids no full scores at these successful settings.

The smaller leaves and larger branch budgets were also tried. They did not produce a faster setting that met the same recall requirement. Wider C keys reduced some work but introduced another tradeoff: the best 24-bit result reached only 73.8% recall within the lookup limits tested. A 20-bit setting reached 95.315%, but took 29.71 ms, compared with 2.97 ms for A at the 95% requirement.

The code checks stored index data against the memory calculation: packed codes and IDs, padded bitplanes for B, and occupied directory entries for C. The measured process memory also includes allocation capacity, temporary arrays and the running program.

Table 4: Index data calculated from the layout and reported by the implementation.

Method	Calculated (MB)	Index data (MB)	Peak RAM (MB)
A	48.454	48.454	190.6
B	88.765	88.765	231.6
C	48.762	48.762	161.7

The parameter sweep answers which tested settings were fastest on this unchanged dataset. For A, increasing the recall requirement from 90% to 99% increases the opened clusters from 353 to 2440, out of 8192. Its median time rises from 1.43 to 8.74 ms. The query vectors stay the same; the extra work comes from opening more clusters. These results concern this dataset and the tested parameter ranges.

4.4 Code references

Run `experiments.fixed_data_comparison` with its `sweep`, `extend` and `repeat` commands. All three read the same arrays; the README gives the full commands and input paths. The search implementations are `cpp/index.cpp` and `cpp/key_index.cpp`. Results, parameter choices, input hashes and raw observations are saved under `results/fixed-data-comparison-2026-09-12/`.

5 Limitations and Next steps

This experiment compares the declared index settings on one unchanged document collection and query set. All methods use the same score, reference answers, RAM budget and CPU thread count. The selected settings are the fastest observed successful choices in that comparison; they are not a proof of the best possible implementation.

The formulas count document scores, mask reads and key lookups. Their time cost depends on memory access, code organization and the machine. Knowing the ideal binary pattern does not by itself tell us how many branches or keys must be visited to recover the reference answers. Those data-dependent counts must be checked rather than assumed.

A useful next step is to expand the parameter range or improve a demonstrated expensive operation while retaining this same benchmark. Any later comparison using a different embedding model, document collection or query set should be reported separately.

6 Conclusion

After allowing each method to choose its own cluster count and search settings, the original scan had the lowest measured time at the 80%, 90%, 95% and 99% recall targets on this unchanged dataset. No tested B or C setting beat A's best setting while meeting the same target. At lower targets, A and B were close; this is a measured comparison, not a claim that A is always best.

The result has a concrete explanation: the successful B and C settings still scored the same documents as A. More selective settings either cost more or missed too many reference results. The comparison uses one document collection and one query set throughout. The comparison must keep the inputs unchanged while giving every method its own parameter choices. The practical test is whether the extra work used to avoid scores costs less than the scores it removes, at the same recall requirement and within the memory budget.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources. <https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation. <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.